Sistem afterschool – schema (FastAPI + PostgreSQL)

Nov 11, 2026 · @darius

Sistem prin care personalul și părinții știu unde se află copiii și dacă au ajuns în siguranță. Fiecare copil are o brățară care se scanează în trei puncte: la urcarea în autobuz, la coborârea la afterschool și la plecare. Ce e marcat cu **(P)** e o presupunere care trebuie confirmată cu clientul.

1. Aplicații

Aplicație	Cine o folosește	Scop
Aplicația Părinte (mobil)	Părinți	Confirmă participarea, vede unde e copilul, primește notificări la fiecare scanare
Aplicația Staff / Scanner (mobil)	Șofer, profesor	Citește brățara (NFC), vede lista copiilor așteptați, funcționează și fără internet
Panoul de administrare (web)	Administrator **(P)**	Gestionează conturi, copii, brățări, trasee și grupe; vede rapoarte de prezență

Cele două aplicații mobile pot fi și **una singură, cu ecrane diferite pe rol**: mai puțin cod și un singur login, dar o aplicație mai încărcată.

2. Backend și bază de date

Componente:

- **FastAPI**: API REST pentru aplicații, plus WebSocket pentru ecranele care trebuie să se actualizeze în timp real.
- **PostgreSQL**: toate datele (copii, trasee, confirmări, scanări).
- **Autentificare**: login cu JWT; rolul utilizatorului decide ce poate vedea și face.
- **Notificări push**: Firebase Cloud Messaging (Android + iOS).
- **Job programat**: verifică periodic scanările lipsă și trimite alerte.

	Avantaje	Dezavantaje
FastAPI	Dezvoltare rapidă și cod puțin · documentație API automată (Swagger), utilă pentru echipa de mobil · validare automată a datelor trimise de aplicații · suportă și WebSocket pentru starea live	Python nu e cel mai rapid limbaj (aici nu contează: timpul se duce pe baza de date și pe rețea) · autentificarea și rolurile le construiești tu · ecosistem mai mic decât .NET pentru proiecte foarte mari
PostgreSQL	Datele sunt relaționale (părinți ↔ copii, trasee ↔ curse), iar cheile străine păstrează integritatea · reguli puse direct în bază (de exemplu o singură scanare de fiecare tip pe zi) · rapoartele de prezență se scriu simplu în SQL · securitate pe rânduri, opțional, pentru „părintele vede doar copiii lui”	Schimbările de structură cer migrări · e mai puțin flexibil decât o bază NoSQL când apar câmpuri noi des · trebuie configurate backup-uri și conexiuni

3. Diagrama arhitecturii

```
flowchart LR
    P[Aplicația Părinte] -->|HTTPS| API
    S[Aplicația Staff / Scanner] -->|HTTPS + offline sync| API
    A[Panou admin web] -->|HTTPS| API
    API[Backend FastAPI<br/>auth, reguli, API] --> DB[(PostgreSQL)]
    API --> J[Job alerte<br/>scanări lipsă]
    J --> DB
    API --> F[Firebase Cloud Messaging]
    J --> F
    F -->|push| P
    F -->|alerte| S
```

O scanare pleacă din aplicația Staff, backend-ul o validează și o salvează, apoi trimite notificarea către părinții copilului.

4. Tabele principale

Tabel	Câmpuri cheie	Relații
users	id (PK), nume, email, telefon, parolă (hash), rol	–
children	id (PK), nume, prenume, data nașterii, grupa	–
parent_child	parent_id (FK users), child_id (FK children), relație (mamă, tată, tutore)	**N-N** părinți ↔ copii
bracelets	id (PK), uid (cod NFC, unic), child_id (FK), status (activă / pierdută)	copil **1-N** brățări în timp, una activă
routes	id (PK), nume, autobuz, driver_id (FK users)	șofer **1-N** trasee
route_children	route_id (FK), child_id (FK), stația de urcare	**N-N** trasee ↔ copii
trips	id (PK), route_id (FK), data, driver_id (FK), status	traseu **1-N** curse (o cursă pe zi)
attendance_confirmations	id (PK), child_id (FK), data, status (vine / nu vine), confirmed_by (FK users)	copil **1-N**, una pe zi
scan_events	id (PK), child_id (FK), bracelet_id (FK), tip (urcare / sosire / plecare), trip_id (FK), scanned_by (FK users), ora, metodă (NFC / manual)	copil **1-N** scanări
notifications	id (PK), user_id (FK), scan_event_id (FK), trimisă la, citită la	scanare **1-N** notificări

În prototipul actual există 6 dintre aceste tabele: `users`, `children` (cu brățara inclusă ca un câmp), `parent_child`, `routes`, `attendance_confirmations` și `scan_events`. Nu sunt încă făcute: `bracelets` ca tabel separat, `route_children`, `trips` și `notifications`.

Tabele opționale (după MVP): `groups` (grupe și profesori), `authorized_pickups` (cine poate lua copilul), `stops` / `route_stops` (stațiile traseului), `buses`, `alerts`, `device_tokens` (telefoanele pentru push), `audit_log`.

```
erDiagram
    USERS ||--o{ PARENT_CHILD : "este părinte"
    CHILDREN ||--o{ PARENT_CHILD : "are părinți"
    CHILDREN ||--o{ BRACELETS : "poartă"
    USERS ||--o{ ROUTES : "conduce"
    ROUTES ||--o{ ROUTE_CHILDREN : "include"
    CHILDREN ||--o{ ROUTE_CHILDREN : "e pe"
    ROUTES ||--o{ TRIPS : "cursă zilnică"
    CHILDREN ||--o{ ATTENDANCE_CONFIRMATIONS : "are"
    CHILDREN ||--o{ SCAN_EVENTS : "are"
    BRACELETS ||--o{ SCAN_EVENTS : "scanată în"
    TRIPS ||--o{ SCAN_EVENTS : "în timpul"
    USERS ||--o{ SCAN_EVENTS : "scanează"
    SCAN_EVENTS ||--o{ NOTIFICATIONS : "generează"
    USERS ||--o{ NOTIFICATIONS : "primește"
```

5. Utilizatori

Rol	Ce poate vedea	Ce poate face
Părinte	Doar copiii lui · starea curentă (în autobuz / la afterschool / plecat) · ora fiecărei scanări · istoricul	Confirmă sau anulează participarea pe ziua respectivă · primește notificări · își actualizează datele de contact
Șofer	Traseul și lista copiilor așteptați azi · cine a confirmat și cine nu · cine lipsește la urcare	Pornește și încheie cursa · scanează copiii la urcare · bifează manual un copil dacă brățara nu merge
Profesor	Copiii din grupa lui · cine e pe drum, cine a ajuns, cine a plecat · cine are voie să ia fiecare copil **(P)**	Scanează sosirea și plecarea · notează cine a luat copilul · marchează un copil adus direct de părinte

Utilizatori propuși, de confirmat cu clientul:

Utilizator propus	De ce
Administrator	Cineva trebuie să creeze conturile, să lege brățările de copii și să facă traseele
Coordonator / director de afterschool	Vede tot afterschool-ul și primește alertele importante (copil nesosit, incident)
Persoană autorizată să ia copilul (bunic, bonă)	Profesorul trebuie să știe cine are voie să ia copilul. Poate fi doar o înregistrare, fără cont
Însoțitor în autobuz	Dacă există, el scanează în locul șoferului, care nu ar trebui să folosească telefonul la volan

6. Ziua unui copil

Pas	Cine acționează	Ce se salvează	Cine e anunțat
1. Confirmarea (seara sau dimineața)	Părintele	`attendance_confirmations`: vine / nu vine	Lista șoferului și a profesorului se actualizează
2. Pornirea cursei	Șoferul	`trips`: cursa de azi pornește; lista copiilor se descarcă în telefon	–
3. Urcarea în autobuz	Șoferul scanează brățara	`scan_events`: urcare	Părintele: „a urcat în autobuz la 12:40”
4. Coborârea la afterschool	Profesorul scanează	`scan_events`: sosire	Părintele: „a ajuns la afterschool”
5. Verificarea automată	Jobul de alerte	Dacă există urcare fără sosire în X minute: `alerts`	Profesorul și coordonatorul
6. Plecarea acasă	Profesorul verifică persoana și scanează	`scan_events`: plecare + cine l-a luat	Părintele: „a plecat cu …”

Pas	Cine acționează	Ce se salvează	Cine e anunțat
7. Sfârșitul zilei	Jobul de alerte	Copiii confirmați, dar nescanați, apar în raport	Coordonatorul

```
sequenceDiagram
    participant P as Părinte
    participant S as Șofer
    participant T as Profesor
    participant B as Backend + DB
    P->>B: Confirmă: vine azi
    S->>B: Pornește cursa
    S->>B: Scanare la urcare
    B-->>P: Notificare: a urcat în autobuz
    T->>B: Scanare la sosire
    B-->>P: Notificare: a ajuns la afterschool
    T->>B: Scanare la plecare (cine l-a luat)
    B-->>P: Notificare: a plecat acasă
```

7. Situații care pot merge prost

Situație	Cum o detectează sistemul	Cum o rezolv
Brățară pierdută sau defectă	Copilul e în listă, dar nu poate fi scanat	Bifare manuală din listă (marcată ca „manual”); adminul blochează brățara și emite una nouă
Fără internet în autobuz	Aplicația nu ajunge la server	Scanările se salvează în telefon și se trimit automat când revine conexiunea; lista se descarcă la pornirea cursei
Copil urcat, dar nescanat la sosire	Urcare fără sosire după X minute de la sosirea autobuzului	Alertă la profesor și coordonator; verificare imediată în autobuz și la afterschool

Situație	Cum o detectează sistemul	Cum o rezolv
Copil scanat, deși părintele a spus că nu vine	Scanare pentru un copil cu „nu vine”	Avertisment pe ecranul celui care scanează și notificare urgentă către părinte
Copil luat de o persoană neautorizată	Persoana nu e pe lista celor autorizați	Profesorul vede lista înainte de plecare; plecarea cere alegerea persoanei; părintele e anunțat cu cine a plecat copilul
Scanare dublă	Aceeași scanare de două ori în aceeași zi	Regulă în baza de date: o singură scanare de fiecare tip pe zi; a doua e ignorată
Telefonul șoferului descărcat sau stricat	Cursa nu trimite scanări	Al doilea dispozitiv (însoțitor, telefon de rezervă) sau profesorul face check-in-ul la sosire; procedură scrisă pentru acest caz
Copil confirmat care nu apare deloc	Confirmat, dar fără nicio scanare până la ora cursei	Alertă la șofer și profesor; părintele e sunat

8. Prima versiune și mai târziu

Starea prototipului la 11 noiembrie 2026 (FastAPI + PostgreSQL): **2 făcute, 4 parțial, 6 nefăcute** din cele 12. În plus față de plan există 62 de teste automate pentru cazuri-limită.

Funcționalitate	Când	De ce	Stare acum
Login și roluri (părinte, șofer, profesor, admin)	**MVP**	Fără ele nu poți folosi date reale despre copii	**Parțial** – rolurile sunt verificate pe server; login-ul e simulat (alegere din listă)
Confirmarea participării	**MVP**	Dă lista copiilor așteptați, pe care se bazează restul	**Făcut** – părintele confirmă sau anulează; anularea e blocată după scanare

Funcționalitate	Când	De ce	Stare acum
Scanare NFC în 3 puncte + bifare manuală	**MVP**	E nucleul sistemului	**Parțial** – cele 3 puncte și bifarea manuală merg; NFC-ul e simulat prin cod
Notificări push la fiecare scanare	**MVP**	E ce așteaptă părinții de la sistem	**Nefăcut** – aplicația se reîmprospătează la 5 secunde, fără notificări
Admin simplu: copii, brățări, trasee	**MVP**	Fără el, datele nu pot fi introduse	**Nefăcut** – datele vin din datele de test
Alertă de scanare lipsă	**MVP**	E partea de siguranță	**Făcut** – „urcat, dar nesosit” apare la profesor și la părinte (fără push)
Lucru offline în aplicația Staff	MVP sau imediat după	Depinde de acoperirea cu internet pe trasee	**Nefăcut**
Persoane autorizate să ia copilul	Versiunea 2	Important, dar se poate începe cu verificare pe hârtie	**Parțial** – se notează cine a luat copilul; nu există încă lista persoanelor autorizate
GPS live pentru autobuz	Mai târziu	Cost și complexitate mai mari, plus confidențialitate	**Nefăcut**
Rapoarte și statistici de prezență	Mai târziu	Utile pentru administrație, nu pentru siguranța zilnică	**Parțial** – părintele vede istoricul ultimelor zile; fără rapoarte pentru administrație
Plăți și facturare, mesaje părinte–profesor	Mai târziu	Nu țin de scopul principal	**Nefăcut**
Mai multe afterschool-uri pe aceeași platformă	Mai târziu	Contează doar dacă produsul se extinde	**Nefăcut**

9. Întrebări pentru client

Funcționalități și roluri

- Cine administrează sistemul: afterschool-ul, școala sau noi? *Decide dacă e nevoie de rolul de administrator și de panoul web.*
- Există însoțitor în autobuz sau doar șoferul? *Decide cine scanează la urcare.*
- Un copil are întotdeauna aceeași grupă și același profesor? *Influențează tabelele de grupe.*

Brățări și dispozitive

- Brățările există deja? Ce tehnologie au (NFC, QR, RFID)? *Decide aplicația de scanare și costul.*
- Se scanează cu telefoanele personalului sau cu dispozitive dedicate? Android, iOS sau ambele? *Influențează dezvoltarea mobilă.*

Notificări

- Ce notificări vor părinții: la fiecare scanare sau doar la probleme? Push, SMS sau ambele? *SMS-ul are cost per mesaj.*
- Cine primește alerta când un copil nu ajunge și în cât timp? *Definește regula de alertă.*

Trasee și autobuze

- Câte autobuze și trasee sunt? Se schimbă des? *Decide cât de complex e modulul de trasee.*
- Copiii urcă de la școli diferite sau din stații? *Influențează tabelele de stații.*
- E nevoie de poziția autobuzului pe hartă? *GPS-ul e un modul separat.*

Preluarea copilului

- Cum se verifică azi cine ia copilul? Există o listă de persoane autorizate? *Decide dacă intră în MVP.*
- Poate pleca un copil singur acasă? *Încă o variantă a pasului de plecare.*

Date personale și GDPR

- Cine e operatorul de date și cât timp se păstrează istoricul? Se stochează date medicale? *Influențează securitatea, consimțământul și ștergerea datelor.*

Scară, buget și termene

- Câți copii, părinți și angajați sunt? Se va extinde la alte afterschool-uri? *Decide infrastructura.*

- Care e bugetul și până când trebuie prima versiune? *Decide ce intră în MVP.*

Rezumat pentru prezentare

Sistemul are trei aplicații: una pentru părinți, una de scanare pentru șofer și profesor, și un panou web de administrare. Toate vorbesc cu un singur backend FastAPI, care salvează datele în PostgreSQL. Fiecare copil e scanat de trei ori pe zi: la urcarea în autobuz, la sosirea la afterschool și la plecare, iar părintele primește notificare la fiecare pas. Partea de siguranță vine din alerte: sistemul observă când un copil a urcat, dar n-a ajuns, sau când a fost confirmat, dar nu apare. Prima versiune acoperă confirmarea, scanarea, notificările și alertele; GPS-ul, rapoartele și extinderea la mai multe afterschool-uri vin ulterior.